



Universidad Politécnica Internacional

Curso: TECNICAS DE PROGRAMACION

Entregable #1

Profesor:

LUIS FELIPE MORA UMAÑA

Integrantes:

-LUIS ALONSO ESPINOZA BONILLA

-DANNEL ROBERTO QUESADA SOLANO

2do Cuatrimestre del 2026

Introducción

Este documento corresponde al Entregable #1 del Proyecto #1 llamado Quiniégol, el cual tiene como propósito planificar el desarrollo inicial de un sistema de gestión de quinielas mundialistas, este documento se enfoca en organizar el proyecto antes de iniciar con su desarrollo ósea, dividiendo el trabajo desde una visión general hasta tareas un poco más pequeñas y manejables.

La estructura de planificación se va a realizar utilizando una jerarquía basada en Epic, Features, Product Backlog Items y tareas de implementación. Esta organización permite visualizar las principales funcionalidades que deberá tener el sistema, así como las actividades necesarias para desarrollarlas de forma ordenada.

Dicha planificación busca facilitar el trabajo del equipo, mejorar el seguimiento de avances mediante Jira y establecer una base clara para el desarrollo posterior del sistema en C#, además, se van a utilizar buenas prácticas como programación orientada a objetos, patrón Modelo-Vista-Controlador, Clean Code, SOLID, manejo de archivos JSON y control de versiones con Git.

Descripción general del proyecto

Quiniégol será un sistema de escritorio desarrollado en C#, orientado a la gestión de quinielas mundialistas. La aplicación permitirá registrar usuarios, administrar partidos, manejar pronósticos, calcular puntajes, mostrar rankings y generar estadísticas relacionadas con los resultados y la participación de los usuarios.

El sistema trabajará con información almacenada en archivos JSON, los cuales contendrán los datos necesarios para la primera iteración del proyecto. Entre estos datos se contemplan usuarios, partidos, selecciones, pronósticos y quinielas privadas.

La aplicación permitirá que los usuarios participen en una o más quinielas, realicen pronósticos antes del inicio de los partidos y acumulen puntos según sus aciertos, el marcador exacto otorgará 5 puntos, mientras que acertar el ganador o empate ganara 2 puntos y también se planea incluir insignias, rankings, timeline de notificaciones y estadísticas visuales dentro del sistema.

Herramientas y tecnologías planificadas

Para el desarrollo y control del proyecto se utilizarán las siguientes herramientas y tecnologías:

Lenguaje de programación: C#.

Interfaz gráfica: Windows Forms.

Arquitectura: Modelo-Vista-Controlador.

Formato de almacenamiento: JSON.

Control de versiones: Git.

Repositorio remoto: GitHub.

Gestión del proyecto: Jira.

Revisión de código: Sonar Analyzer.

Paradigma principal: Programación Orientada a Objetos.

El uso de estas herramientas permitirá organizar el proyecto de forma más clara, mantener control sobre los cambios realizados, facilitar el trabajo colaborativo y aplicar buenas prácticas de desarrollo.

Estructura general de planificación

La planificación del proyecto se dividirá de la siguiente manera:

Epic: representa el proyecto general que se desea construir.

Feature: representa una funcionalidad grande del sistema.

Product Backlog Item: representa una necesidad específica dentro de una funcionalidad.

Tareas de implementación: representan actividades pequeñas y concretas necesarias para completar cada PBI.

Esta estructura permite dividir un proyecto grande en partes más simples, facilitando el análisis, la asignación de trabajo y el seguimiento del avance dentro de Jira.

Epic del proyecto

Epic 1: Sistema Quiniégol

El sistema Quiniégol será una aplicación para la gestión de quinielas mundialistas, el objetivo será permitir que los usuarios puedan registrarse, participar en quinielas públicas o privadas, realizar pronósticos de partidos, consultar rankings, revisar estadísticas y recibir notificaciones relacionadas con los eventos del sistema.

El proyecto se desarrollará considerando principios de programación orientada a objetos, Clean Code, arquitectura MVC y buenas prácticas que permitan que el sistema sea mantenible, extensible y preparado para futuras iteraciones.

Features, Product Backlog Items y tareas de implementación

Feature 1.1: Gestión de usuarios

Esta funcionalidad permitirá registrar y administrar los usuarios que utilizarán la aplicación, cada usuario podrá tener información básica como nombre, país preferido, puntos acumulados, pronósticos realizados, quinielas a las que pertenece e insignias obtenidas.

PBI 1.1.1: Registro de usuarios

Se necesitará una opción para registrarse dentro de la aplicación, ingresando la información básica necesaria para participar en las quinielas.

Criterio de aceptación:

El sistema permite registrar un nuevo usuario con sus datos principales y almacena la información correctamente en un archivo JSON, El sistema validará que los datos obligatorios no estén vacíos y evita registros duplicados.

Tareas de implementación:

Crear la clase Usuario.

Definir propiedades como id, nombre, país preferido, puntos e insignias.

Crear formulario de registro de usuario en Windows Forms.

Implementar validaciones de campos obligatorios.

Crear servicio para guardar usuarios en archivo JSON.

Probar el registro de usuarios con datos válidos e inválidos.

PBI 1.1.2: Consulta y visualización de usuarios

Se necesitará visualizar la lista de usuarios registrados para consultar su información básica y puntaje acumulado.

Criterio de aceptación:

El sistema muestra en una pantalla la lista de usuarios registrados, incluyendo nombre, país preferido y puntos acumulados.

Tareas de implementación:

Crear método para leer usuarios desde el archivo JSON.

Crear pantalla de consulta de usuarios.

Mostrar la información en una tabla o componente visual.

Agregar opción de actualización de la lista.

Validar que el sistema muestre un mensaje cuando no existan usuarios registrados.

Feature 1.2: Gestión de selecciones y partidos

Esta funcionalidad permitirá administrar la información de las selecciones participantes y los partidos del torneo. Los partidos tendrán datos como equipos participantes, fecha, hora, estado, marcador y anotadores.

PBI 1.2.1: Carga de selecciones desde archivo JSON

Se necesitará cargar la información de las selecciones desde un archivo JSON para que pueda ser utilizada en partidos, pronósticos y estadísticas.

Criterio de aceptación:

El sistema carga correctamente las selecciones desde un archivo JSON y permite consultarlas dentro de la aplicación.

Tareas de implementación:

Crear la clase Seleccion.

Definir propiedades como id, nombre, grupo y ruta de bandera.

Crear archivo JSON inicial de selecciones.

Implementar servicio de lectura de selecciones.

Mostrar las selecciones disponibles en la interfaz.

Validar errores en caso de archivo inexistente o datos incorrectos.

PBI 1.2.2: Gestión de partidos

Se necesitará consultar los partidos disponibles, incluyendo selecciones participantes, fecha, hora, estado y marcador.

Criterio de aceptación:

El sistema muestra los partidos cargados desde JSON y permite identificar si un partido está pendiente, en curso o finalizado.

Tareas de implementación:

Crear la clase Partido.

Definir propiedades como id, selección local, selección visitante, fecha, hora, estado y goles.

Crear archivo JSON de partidos.

Implementar lectura de partidos desde JSON.

Crear pantalla para visualizar partidos.

Validar el estado del partido según la fecha simulada del sistema.

PBI 1.2.3: Registro de resultados y anotadores

Se registrarán los resultados finales de los partidos y los anotadores correspondientes.

Criterio de aceptación:

El sistema permite actualizar el resultado de un partido finalizado y mostrar los anotadores una vez que el partido haya terminado.

Tareas de implementación:

Crear clase o estructura para representar anotadores.

Agregar lista de anotadores a la clase Partido.

Crear pantalla para registrar o editar resultados.

Guardar los resultados actualizados en JSON.

Mostrar anotadores en la vista de partidos finalizados.

Validar que no se registren resultados incompletos.

Feature 1.3: Gestión de quinielas

Esta funcionalidad permitirá crear y administrar quinielas públicas y privadas, cada quiniela tendrá integrantes, posiciones, ranking interno y timeline de notificaciones.

PBI 1.3.1: Creación de quinielas privadas

Se crearán quinielas privadas para competir con otros usuarios dentro de un grupo específico.

Criterio de aceptación:

El sistema permite crear una quiniela privada con nombre, descripción e integrantes. La información se almacena correctamente en JSON.

Tareas de implementación:

Crear la clase Quiniela.

Definir propiedades como id, nombre, tipo, integrantes y ranking interno.

Crear formulario para registrar quinielas.

Permitir seleccionar usuarios integrantes.

Guardar quinielas privadas en archivo JSON.

Crear al menos 5 quinielas privadas iniciales para pruebas.

PBI 1.3.2: Asociación de usuarios a quinielas

Se necesitará pertenecer a una o más quinielas para participar en diferentes rankings.

Criterio de aceptación:

El sistema permite asociar un usuario a una o varias quinielas y refleja correctamente esa información en la consulta de quinielas.

Tareas de implementación:

Crear método para agregar usuarios a una quiniela.

Validar que el usuario no se agregue dos veces a la misma quiniela.

Actualizar el archivo JSON de quinielas.

Mostrar los integrantes de cada quiniela.

Probar usuarios pertenecientes a múltiples quinielas.

Feature 1.4: Gestión de pronósticos

Esta funcionalidad permitirá que los usuarios registren sus pronósticos para los partidos antes de que inicien, una vez que el partido esté en curso el sistema no deberá aceptar nuevos pronósticos ni modificaciones.

PBI 1.4.1: Registro de pronósticos

Se necesitará registrar el marcador pronosticado para un partido antes de que inicie.

Criterio de aceptación:

El sistema permite registrar un pronóstico válido para un partido pendiente y lo guarda en el archivo JSON correspondiente.

Tareas de implementación:

Crear la clase Pronostico.

Definir propiedades como usuario, partido y goles pronosticados.

Crear formulario para registrar pronósticos.

Cargar lista de partidos disponibles.

Guardar pronósticos en JSON.

Validar que el marcador ingresado sea numérico y no negativo.

PBI 1.4.2: Bloqueo de pronósticos cuando inicia el partido

Se necesitará bloquear el registro o modificación de pronósticos cuando un partido ya inició, utilizando una fecha simulada del sistema.

Criterio de aceptación:

El sistema no permite crear ni modificar pronósticos de partidos que estén en curso o finalizados.

Tareas de implementación:

Crear configuración de fecha simulada.

Implementar método para comparar la fecha simulada con la fecha del partido.

Bloquear controles de registro cuando el partido no esté pendiente.

Mostrar mensaje informativo al usuario.

Probar partidos pendientes, en curso y finalizados.

PBI 1.4.3: Historial de pronósticos

Se necesitará consultar el historial de pronósticos realizados para revisar participaciones anteriores.

Criterio de aceptación:

El sistema muestra los pronósticos realizados por usuario, incluyendo partido, marcador pronosticado, resultado real y puntos obtenidos si ya fue calculado.

Tareas de implementación:

Crear método para filtrar pronósticos por usuario.

Crear pantalla de historial de pronósticos.

Mostrar resultado real cuando el partido haya finalizado.

Mostrar puntos obtenidos por cada pronóstico.

Validar que se muestre un mensaje cuando el usuario no tenga pronósticos.

Feature 1.5: Cálculo de puntajes y rankings

Esta funcionalidad permitirá calcular los puntos obtenidos por los usuarios y mostrar rankings globales y privados según su desempeño.

PBI 1.5.1: Cálculo de puntos por pronóstico

Se necesitará calcular los puntos obtenidos por cada usuario según sus aciertos en los partidos.

Criterio de aceptación:

El sistema asigna 5 puntos cuando el usuario acierta el marcador exacto y 2 puntos cuando acierta el ganador o empate.

Tareas de implementación:

Crear servicio de cálculo de puntajes.

Comparar pronóstico contra resultado real.

Asignar puntos según las reglas definidas.

Actualizar puntos acumulados del usuario.

Guardar resultados del cálculo en JSON.

Probar casos de marcador exacto, ganador correcto, empate correcto y pronóstico incorrecto.

PBI 1.5.2: Ranking global

Se necesitará consultar el ranking global para conocer las posiciones generales de todos los participantes.

Criterio de aceptación:

El sistema muestra una lista ordenada de usuarios según puntos acumulados, de mayor a menor.

Tareas de implementación:

Crear método para ordenar usuarios por puntos.

Crear pantalla de ranking global.

Mostrar posición, nombre, país preferido y puntos.

Actualizar ranking después del cálculo de resultados.

Validar empates en puntaje.

PBI 1.5.3: Ranking por quiniela privada

Se necesitará consultar el ranking interno de esa quiniela.

Criterio de aceptación:

El sistema muestra únicamente los usuarios pertenecientes a la quiniela seleccionada, ordenados por puntaje.

Tareas de implementación:

Crear método para filtrar usuarios por quiniela.

Calcular posiciones dentro de cada quiniela.

Crear pantalla de ranking privado.

Permitir seleccionar la quiniela a consultar.

Mostrar mensaje si una quiniela no tiene integrantes.

Feature 1.6: Insignias y reconocimientos

Esta funcionalidad permitirá asignar insignias a los usuarios según diferentes condiciones, como primer lugar global, primer lugar en quiniela privada, rey de los empates, racha de aciertos y posiciones negativas.

PBI 1.6.1: Asignación de insignias positivas

Se necesitará asignar insignias positivas a los usuarios que destaquen por su rendimiento.

Criterio de aceptación:

El sistema asigna insignias a usuarios que cumplan condiciones como primer lugar global, primer lugar en quiniela privada, rey de los empates o racha de más de 10 aciertos.

Tareas de implementación:

Crear clase Insignia.

Definir tipos de insignias positivas.

Crear reglas para asignación automática.

Asociar insignias al usuario.

Mostrar insignias en la pantalla del perfil o ranking.

Probar condiciones de asignación.

PBI 1.6.2: Asignación de insignias de vergüenza

Descripción:

Como sistema, se necesita asignar insignias negativas o de vergüenza para usuarios con bajo rendimiento en quinielas o ranking global.

Criterio de aceptación:

El sistema identifica al peor jugador de la liga o del ranking global y le asigna la insignia correspondiente.

Tareas de implementación:

Definir tipos de insignias de vergüenza.

Crear método para identificar usuarios con menor puntaje.

Asociar insignias negativas al usuario correspondiente.

Mostrar insignias en el ranking o perfil.

Validar que la insignia cambie si cambian los puntajes.

Feature 1.7: Timeline de notificaciones

Esta funcionalidad permitirá que cada quiniela tenga un timeline con mensajes importantes sobre eventos del sistema, como aciertos, rachas, cambios de líder e insignias de vergüenza.

PBI 1.7.1: Registro de eventos en timeline

Se necesitará generar mensajes automáticos dentro del timeline de cada quiniela cuando ocurran eventos importantes.

Criterio de aceptación:

El sistema registra mensajes como aciertos de marcador, nuevas rachas, nuevo líder o mensajes de vergüenza dentro de la quiniela correspondiente.

Tareas de implementación:

Crear clase NotificacionTimeline.

Definir tipos de eventos del timeline.

Crear método para generar mensajes automáticos.

Asociar notificaciones a una quiniela.

Guardar timeline en JSON.

Mostrar timeline en pantalla.

PBI 1.7.2: Visualización del timeline por quiniela

Se necesitará visualizar el timeline de eventos para conocer la actividad reciente del grupo.

Criterio de aceptación:

El sistema muestra una lista de notificaciones relacionadas con la quiniela seleccionada, ordenadas por fecha.

Tareas de implementación:

Crear pantalla de timeline.

Cargar notificaciones desde JSON.

Ordenar notificaciones por fecha.

Permitir seleccionar quiniela.

Mostrar mensaje si no existen eventos registrados.

Feature 1.8: Estadísticas del sistema

Esta funcionalidad permitirá visualizar estadísticas generales del sistema en tiempo de consulta, según rangos de fechas específicos.

PBI 1.8.1: Estadística de equipo más apostado

Se necesitará consultar cuál selección ha sido más pronosticada como ganadora dentro de un rango de fechas.

Criterio de aceptación:

El sistema muestra el equipo más apostado según los pronósticos registrados y el rango de fechas seleccionado.

Tareas de implementación:

Crear servicio de estadísticas.

Filtrar pronósticos por rango de fechas.

Contabilizar selecciones más elegidas.

Mostrar resultado en pantalla.

Validar fechas inválidas o vacías.

PBI 1.8.2: Estadística de resultado más repetido

Se necesitará consultar cuál marcador ha sido más repetido en los pronósticos.

Criterio de aceptación:

El sistema muestra el marcador pronosticado con mayor frecuencia dentro del rango de fechas indicado.

Tareas de implementación:

Agrupar pronósticos por marcador.

Contar repeticiones por resultado.

Identificar el marcador más repetido.

Crear pantalla de visualización.

Validar que existan datos suficientes para calcular la estadística.

PBI 1.8.3: Estadísticas de aciertos y participación

Se necesitará consultar estadísticas como partido con más aciertos, usuario con más aciertos, partidos con más pronósticos y promedio de goles.

Criterio de aceptación:

El sistema muestra estadísticas generales calculadas a partir de partidos, resultados y pronósticos registrados.

Tareas de implementación:

Crear métodos para calcular aciertos por partido.

Crear método para identificar usuario con más aciertos.

Crear método para identificar partido con más pronósticos.

Crear cálculo de promedio de goles.

Mostrar estadísticas en pantalla.

Validar rangos de fechas.

Feature 1.9: Carga y persistencia de datos JSON

Esta funcionalidad permitirá almacenar y recuperar la información del sistema mediante archivos JSON.

PBI 1.9.1: Creación de archivos JSON iniciales

Se necesitará crear los archivos JSON iniciales que contendrán usuarios, partidos, pronósticos, selecciones y quinielas.

Criterio de aceptación:

El sistema cuenta con archivos JSON válidos que contienen la información mínima requerida para realizar pruebas de funcionamiento.

Tareas de implementación:

Crear archivo usuarios.json.

Crear archivo partidos.json.

Crear archivo pronosticos.json.

Crear archivo selecciones.json.

Crear archivo quinielas.json.

Incluir al menos 40 usuarios con diferentes resultados y pronósticos.

Incluir mínimo 5 quinielas privadas.

PBI 1.9.2: Lectura y escritura de archivos JSON

Se necesitará leer y escribir datos desde archivos JSON para mantener la información actualizada.

Criterio de aceptación:

El sistema puede cargar datos al iniciar y guardar cambios cuando se registran usuarios, quinielas, pronósticos o resultados.

Tareas de implementación:

Crear servicio genérico o específico para manejo de JSON.

Implementar lectura de archivos.

Implementar escritura de archivos.

Manejar excepciones por archivos inexistentes o formato inválido.

Probar guardado y lectura de datos modificados.

Feature 1.10: Arquitectura MVC y organización del proyecto

Esta funcionalidad permitirá organizar el sistema en capas, separando la lógica, los datos y la interfaz gráfica.

PBI 1.10.1: Definición de estructura MVC

Se necesitará definir una estructura de carpetas y clases basada en MVC para mantener el proyecto ordenado y facilitar futuras modificaciones.

Criterio de aceptación:

El proyecto cuenta con una estructura clara que separa modelos, vistas, controladores y servicios.

Tareas de implementación:

Crear carpeta Models.

Crear carpeta Views.

Crear carpeta Controllers.

Crear carpeta Services.

Crear carpeta Data.

Definir responsabilidades de cada capa.

Documentar brevemente la estructura del proyecto.

PBI 1.10.2: Comunicación entre vistas, controladores y servicios

Se necesitará que las vistas se comuniquen con controladores y servicios sin mezclar directamente la lógica de negocio con la interfaz.

Criterio de aceptación:

Las pantallas de Windows Forms utilizan controladores o servicios para ejecutar operaciones, evitando concentrar toda la lógica en los formularios.

Tareas de implementación:

Crear controladores para usuarios, partidos, quinielas y pronósticos.

Crear servicios para cálculo de puntajes, estadísticas y manejo de JSON.

Conectar formularios con controladores.

Evitar lógica compleja dentro de eventos de botones.

Revisar estructura con Sonar Analyzer.

Organización del trabajo en Jira

Para dar seguimiento al proyecto, se utilizará Jira como herramienta de gestión, en Jira se crearán las tareas principales siguiendo la estructura definida en este documento.

La organización propuesta será la siguiente:

Crear un proyecto en Jira llamado Quiniegol.

Registrar el Epic principal: Sistema Quiniegol.

Crear las Features como funcionalidades principales.

Crear los PBIs como historias o tareas específicas.

Agregar subtareas de implementación para cada PBI.

Asignar responsables entre los integrantes del equipo.

Actualizar el estado de las tareas según su avance.

Los estados de trabajo propuestos son:

Pendiente

En proceso

En revisión

Finalizado

Esta organización permitirá tener mayor control sobre el avance del proyecto y facilitará la revisión del trabajo realizado por cada integrante.

Consideraciones de calidad y buenas prácticas

Durante el desarrollo del sistema se tomarán en cuenta las siguientes buenas prácticas:

Utilizar nombres significativos para clases, métodos, variables y archivos.

Mantener métodos cortos y enfocados en una sola responsabilidad.

Aplicar principios SOLID cuando sea posible.

Separar la lógica de negocio de la interfaz gráfica.

Utilizar encapsulamiento en las clases del modelo.

Implementar manejo de excepciones para errores de lectura, escritura y validación.

Mantener una estructura ordenada del repositorio.

Realizar commits frecuentes en Git.

Utilizar Sonar Analyzer para detectar posibles problemas de calidad en el código.

Evitar duplicación innecesaria de código.

Diseñar el sistema pensando en futuras modificaciones para una segunda iteración.

Estas prácticas ayudarán a que el sistema sea más mantenible, escalable y robusto.

Conclusiones / Análisis de aprendizaje

Este entregable #1 permite identificar las principales entidades del sistema desde la etapa de planificación por lo cual será más sencillo diseñar una estructura orientada a objetos y aplicar el patrón MVC de manera adecuada.

Además, el uso de Jira permitirá llevar un mejor control del avance del proyecto, asignar responsabilidades y verificar el cumplimiento de las tareas. Por otra parte, el uso de Git facilitará el control de versiones y el trabajo colaborativo entre los integrantes del equipo.

Como aprendizaje principal, se destaca que esta será la primera vez que se utiliza una herramienta como Jira de manera "real" por lo que será una nueva experiencia de la cual aprender.